

Relatório de Laboratório

Laboratório de Experimentação de Software — Modelo/Template de Relatório

Este documento é um MODELO válido para qualquer um dos 5 laboratórios da disciplina (Lab01 a Lab05). Os parágrafos em itálico cinza/verde, iniciados por “ORIENTAÇÃO”, explicam o que cada subseção deve conter — apague-os e escreva o conteúdo real do grupo no lugar. Textos entre colchetes [assim] indicam onde inserir informação específica do seu grupo/laboratório.

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Daniilo Maia
Laboratório	[Lab01 — Características de repositórios populares]
Grupo (trio)	Pedro Luis · Enrico Bessa · Pericles Pires
Link do repositório / GitHub Projects	https://github.com/PedroL10/Lab01Exp
Data de entrega	27/08/2026

1. Introdução

Repositórios populares no GitHub costumam ser usados como referência implícita de qualidade e boas práticas de engenharia de software, mas nem sempre fica claro quais características eles de fato compartilham. Este laboratório investiga essa questão de forma empírica, coletando métricas dos 1.000 repositórios com mais estrelas do GitHub via API GraphQL.

Perguntas que esta seção deve responder ao leitor

- Qual problema está sendo investigado, e por que ele importa (para a engenharia de software, para o mercado, ou para o próprio grupo)?
- Quais são as Questões de Pesquisa do enunciado (numeradas RQ1, RQ2, ...)?
- Quais as hipóteses informais do grupo para cada RQ, antes de olhar os dados (quando aplicável ao laboratório)?
- Quais RQs, métricas ou variáveis o grupo está propondo além do enunciado (resumo de 1 linha cada — os 30% de inovação)?
- =====
- RQ01 - Sistemas populares são maduros/antigos?
- RQ02 - Sistemas populares recebem muita contribuição externa?
- RQ03 — Sistemas populares lançam releases com frequência?
- RQ04 — Sistemas populares são atualizados com frequência?
- RQ05 — Sistemas populares são escritos nas linguagens mais populares?
- RQ06 — Sistemas populares possuem alto percentual de issues fechadas?
- RQ07 — Sistemas escritos em linguagens mais populares recebem mais contribuição, lançam mais releases e são atualizados com mais frequência?
- RQ08 (inovação do grupo) — Repositórios sem linguagem primária detectável têm indicadores de manutenção piores que a média? (detalhes na seção 3.6)

2. Contexto

Este é o Lab01 da disciplina, primeiro laboratório do semestre — não depende de dados de laboratórios anteriores. O objeto de estudo são os 1.000 repositórios com mais estrelas do GitHub, coletados via API GraphQL oficial

3. Metodologia

ORIENTAÇÃO: Esta é a seção mais longa do relatório e a que mais evidencia o trabalho real do grupo. Ela tem seis subseções — as cinco primeiras cobrem principalmente os 70% do enunciado; a última (Inovações) é onde os 30% de contribuição própria do grupo devem ficar explícitos e fáceis de identificar na correção.

3.1 Principais Desafios

ORIENTAÇÃO: Relate as dificuldades técnicas e metodológicas reais enfrentadas pelo grupo — não uma lista de trivialidades já resolvidas, e sim decisões difíceis de fato. Exemplos típicos, conforme o laboratório: limite de taxa (rate limit) da API do GitHub ao consultar milhares de repositórios ou workflow runs (Lab01/Lab03); paginação de grandes volumes de dados; ausência de histórico de mudança de status consultável via API no GitHub Projects, exigindo snapshots manuais recorrentes (todos os laboratórios); dificuldade de padronizar katas de dificuldade equivalente e evitar memorização de soluções pela IA (Lab02); ambiguidade na definição operacional de uma métrica, como lead time (Lab03); dados incompletos ou repositórios sem GitHub Actions habilitado (Lab03) -

=====

Custo de processamento da API do GitHub: solicitar muitos repositórios de uma vez (50-100) com campos aninhados (issues, releases, pull requests) excedia o limite de custo da API e retornava erro 502/504, provavelmente por causa de repositórios com trackers de issues/PRs muito grandes. Resolvido com paginação em lotes pequenos (10 repositórios por requisição) e retry com backoff exponencial para erros 502/503/504.

- Limite de 1.000 resultados da Search API do GitHub: tanto a API REST quanto a GraphQL limitam buscas a no máximo 1.000 resultados totais, independente da paginação — confirmado na prática ao coletar exatamente 1.000 repositórios sem erro.

- Certificado SSL na rede da instituição: pip install e as chamadas à API falhavam por erro de certificado SSL não confiável na rede do laboratório. Resolvido instalando pip-system-certs, que faz o Python usar o repositório de certificados do próprio Windows.

- Divergência entre fontes de contagem de PRs aceitas: ao validar manualmente pull_requests_merged_total via API REST de busca (/search/issues), os valores ficaram levemente diferentes (menos de 1,5%) dos obtidos pela query GraphQL (pullRequests(states: MERGED)). Explicado pela API de busca usar um índice eventualmente consistente, enquanto a query GraphQL consulta a fonte primária diretamente — por isso o projeto usa a GraphQL como fonte oficial.

- Ausência de histórico de status no GitHub Projects: exigiu snapshots manuais recorrentes via script GraphQL (ver Issue de snapshot), já que a API não expõe histórico de mudança de coluna.

3.2 Tomadas de Decisão

- Linguagem do projeto: Python, escolhido pela boa integração com análise de dados nas próximas sprints (RQs 03-07, visualizações).

- Fonte primária de PRs aceitas: GraphQL (`pullRequests(states: MERGED) { totalCount }`) em vez da API de busca, por ser mais confiável (ver 3.1).
- Tamanho de lote (page size) na paginação: fixado em 10 repositórios por requisição — testado empiricamente; lotes maiores (25-100) causavam erro de custo da API em repositórios com trackers muito grandes.
- Critério de amostra: `search(query: "stars:>1 sort:stars-desc", type: REPOSITORY)` — exclui repositórios com 0-1 estrela (ruído) e ordena por estrelas decrescente.
- Limite de WIP na coluna Doing: 3 cartões — um por integrante ativo no board (trio), garantindo que cada pessoa tenha no máximo uma tarefa em andamento por vez, evitando dispersão de foco e cartões abertos sem responsável dedicado simultâneo.
- Formato de saída por sprint: JSON na Sprint 1 (100 repositórios, inspeção/validação), CSV na Sprint 2 (1.000 repositórios, formato exigido pelo enunciado para a base final).

[conteúdo do grupo — substituir este texto]

3.3 Etapas

ORIENTAÇÃO: Descreva o processo de desenvolvimento em sprints, seguindo a estrutura do enunciado (ex.: Lab0XS01, S02, S03 + Relatório Final), com o que foi efetivamente entregue em cada uma e quem (qual integrante) foi responsável por qual parte — a correção do professor é feita a partir do board (GitHub Projects), então a divisão aqui deve refletir os Assignees reais das Issues, não uma divisão apenas narrativa. Inclua também a subseção “Configuração do processo” exigida em todos os laboratórios: as colunas do board (mínimo Backlog → To Do → Doing → Review → Done), a política de limite de WIP em uso, e uma captura de tela (print) do board ao final do laboratório, mostrando o fluxo real de trabalho do grupo.

Sprint S01:

- Autenticação GraphQL configurada — Responsável: Pedro Luis— Issue #1
- Query base para 100 repositórios — Responsável: Pedro Luis— Issue #2
- Validação de campos RQ01+RQ02 (amostra) — Responsável: Pedro Luis— Issue #3
- Validação de campos RQ03+RQ04 (amostra) — Responsável: Enrico Bessa — Issue #4 [PENDENTE]
- Validação de campos RQ05+RQ06 (amostra) — Responsável: Pericles Pires — Issue #5 [PENDENTE]
- Integração da coleta final (100 repositórios) + registro em JSON — Responsável: Pedro Luis— Issues #6 e #7

Sprint S02:

- Paginação + exportação de 1.000 repositórios em CSV — Responsável: Pedro Luis— Issue #8
- Validação RQ01+RQ02 (1.000 repositórios) + hipótese informal — Responsável: Pedro Luis— Issue #10

- Validação RQ03+RQ04 (1.000 repositórios) + hipótese informal — Responsável: Integrante B — Issue [PREENCHER Nº] [PENDENTE]

- Validação RQ05+RQ06 (1.000 repositórios) + hipótese informal — Responsável: Integrante C — Issue [PREENCHER Nº] [PENDENTE]

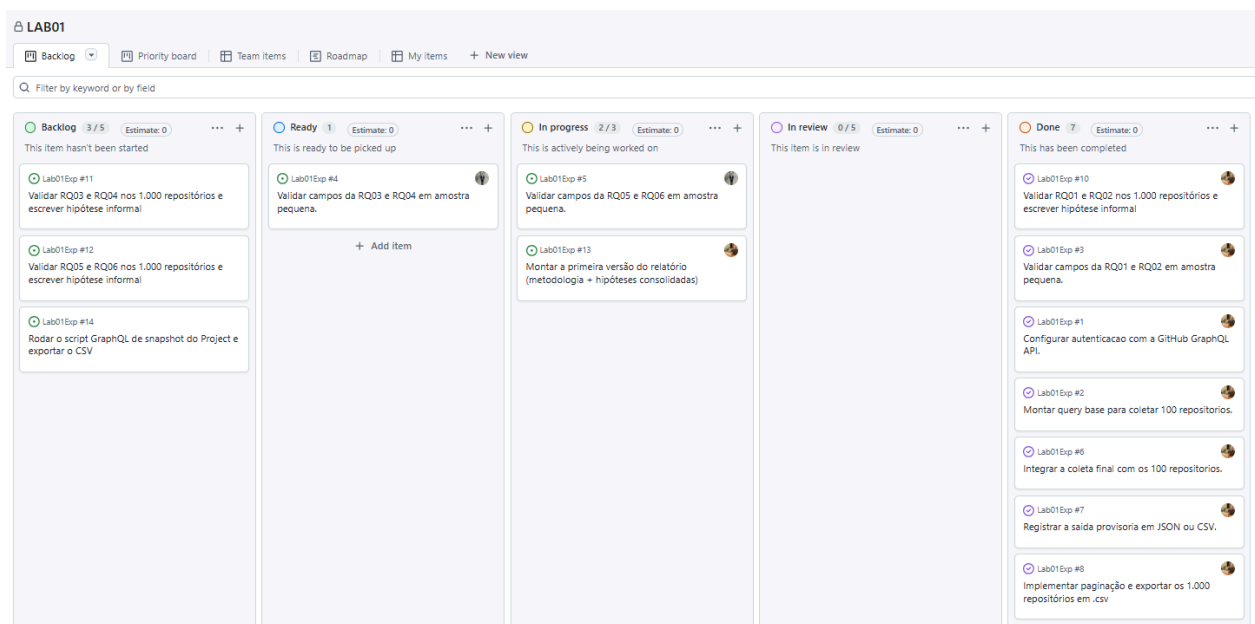
- Primeira versão do relatório — Responsável: Pedro Luis — Issue [PREENCHER Nº 13]

- Snapshot do Project exportado em CSV — Responsável: Você — Issue [PREENCHER Nº]

Configuração do processo:

- Colunas do board: Backlog → To Do → Doing → Review → Done.

- Limite de WIP em Doing: 3 cartões (um por integrante ativo no board).



[Tabela ou linha do tempo com Sprint | Entregas | Responsável(is) | Issues (nº)]

Sugestão: insira aqui o print do quadro Kanban (GitHub Projects) mencionado na orientação acima.

3.4 Ferramentas

- Python 3.11

- requests (cliente HTTP genérico, consumindo a query GraphQL escrita pelo próprio grupo)

- python-dotenv (variáveis de ambiente)

- pip-system-certs (compatibilidade de certificados SSL)
- API GraphQL oficial do GitHub (<https://api.github.com/graphql>)
- GitHub Projects (v2) — <https://github.com/users/PedroL10/projects/1/views/1>

[conteúdo do grupo — substituir este texto]

3.5 Tabela de Métricas

ORIENTAÇÃO: Construa uma tabela relacionando cada Questão de Pesquisa à métrica correspondente, sua definição operacional exata (a fórmula ou regra de cálculo — não basta o nome) e a ferramenta/fonte usada para coletá-la. Isso é o que garante que o laboratório seja reproduzível por outro grupo. A primeira linha abaixo é um exemplo ilustrativo (baseado no Lab01); substitua pelas RQs e métricas do seu laboratório.

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data atual – data de criação do repositório	Dias/anos	Script GraphQL (API do GitHub)
RQ02	Total de PRs aceitas	pullRequests(states: MERGED) { totalCount }	quantidade/contagem	Script GraphQL (API do GitHub)
RQ03	Total de releases	releases { totalCount }	Quantidade/contagem	Script GraphQL (API do GitHub)
RQ04	Tempo até a última atualização	Data atual (UTC) – pushedAt (data do último push de código) Dias	Dias	Script GraphQL (API do GitHub) — campo pushedAt
RQ05	Linguagem primária do repositório	primaryLanguage { name }, comparada ao ranking do GitHub Octoverse 2024 Categórica (nome da linguagem)	Categórica (nome da linguagem)	Script GraphQL (API do GitHub) + GitHub Octoverse 2024 (referência externa)
RQ06	% de issues fechadas	100 × issues(states: CLOSED) { totalCount } / issues { totalCount }	Percentual (%)	Script GraphQL (API do GitHub)
RQ07	RQ02/RQ03/RQ04 segmentadas por linguagem	Agrupamento das métricas de RQ02, RQ03 e RQ04 por primaryLanguage, usando a mediana de cada grupo	Comparativo (mediana por linguagem)	Análise sobre os dados já coletados (analyze_metrics.py)
RQ08	Indicadores de manutenção com e sem linguagem primária	Mediana de RQ02/RQ03/RQ04 no grupo primaryLanguage = null ("Não informado") comparada à mediana geral da amostra	Comparativo (mediana, com/sem linguagem)	Análise sobre os dados já coletados (analyze_metrics.py, campo RQ07_por_linguagem["Nao informado"])

3.6 Inovações Propostas pelo Grupo (30% da nota)

RQ08 (proposta pelo grupo) — Repositórios sem linguagem primária detectável têm indicadores de manutenção piores que a média?

Ao segmentar por linguagem para responder a RQ07, notamos que o grupo "Não informado" (87 repositórios, 8,7% da amostra — em geral listas "awesome" e materiais de referência) se destaca por indicadores muito abaixo da mediana geral: mediana de 129 PRs aceitas (vs. 768 da amostra geral), mediana de 0 releases (vs. 39 geral) e mediana de 181 dias sem atualização (vs. 5 dias geral).

Isso sugere que repositórios sem uma linguagem de programação dominante — tipicamente conteúdo curado ou documentação, não software ativo — são populares, mas usam o GitHub de forma fundamentalmente diferente dos demais: menos como projeto de software em desenvolvimento contínuo, mais como referência estática. Esta é uma métrica adicional, não pedida no enunciado, cujo resultado aprofunda a leitura das RQ05 e RQ07: nem todo repositório popular no GitHub representa "software" no sentido de desenvolvimento ativo.

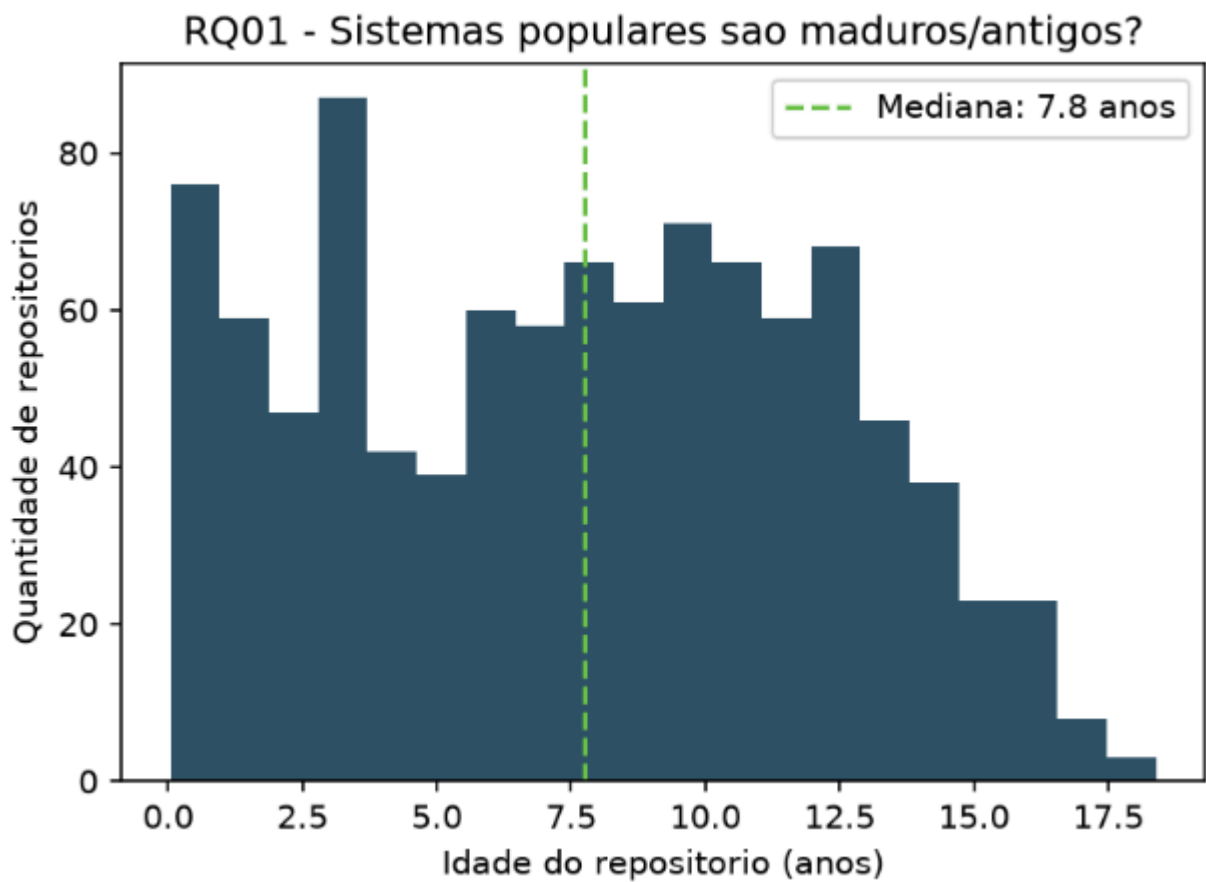
4. Resultados

4.1 Coleta de Dados

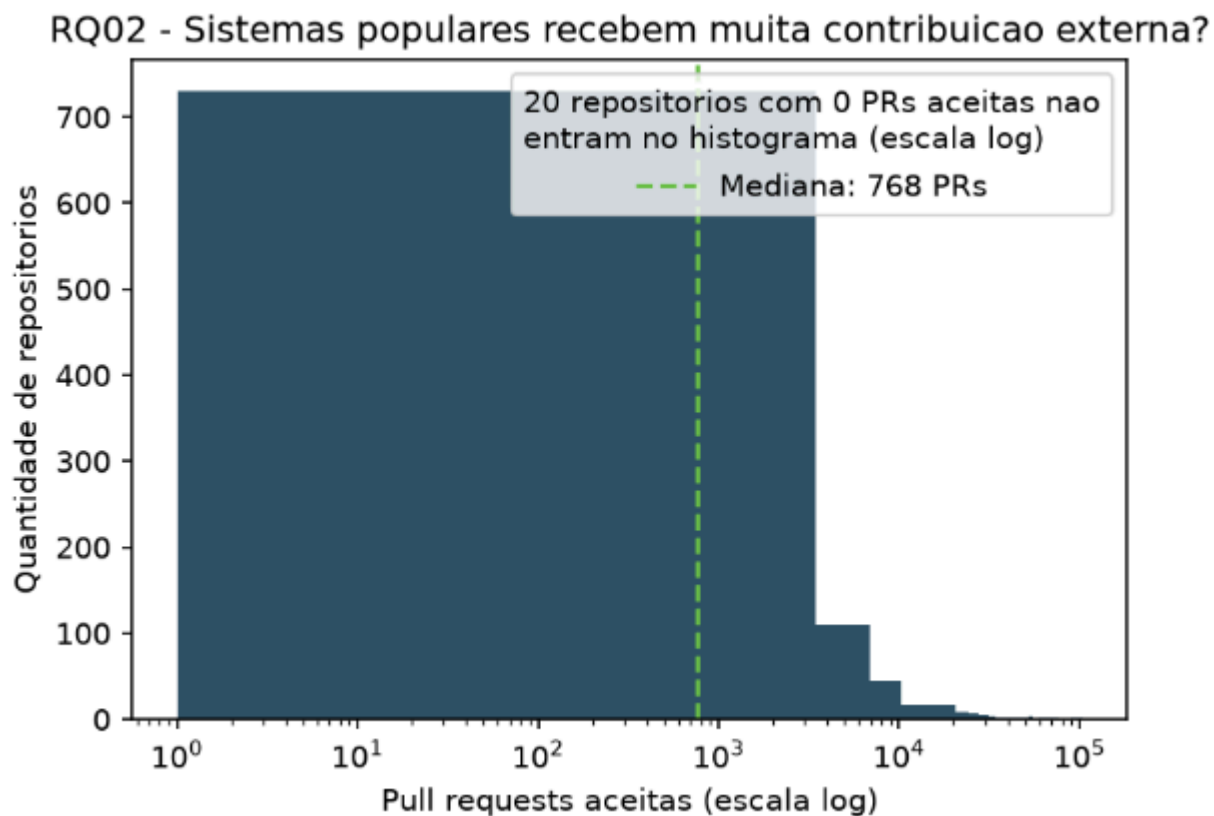
)Para RQ01 e RQ02, os 1.000 repositórios têm `created_at` e `pull_requests_merged_total` completos, sem valores ausentes. Para RQ07, a segmentação por linguagem resultou em 44 linguagens distintas; para evitar medianas pouco confiáveis por amostra pequena, o gráfico comparativo (seção 4.2) considera apenas linguagens com pelo menos 10 repositórios na amostra.

4.2 Visualização Gráfica

RQ01 — Sistemas populares são maduros/antigos?

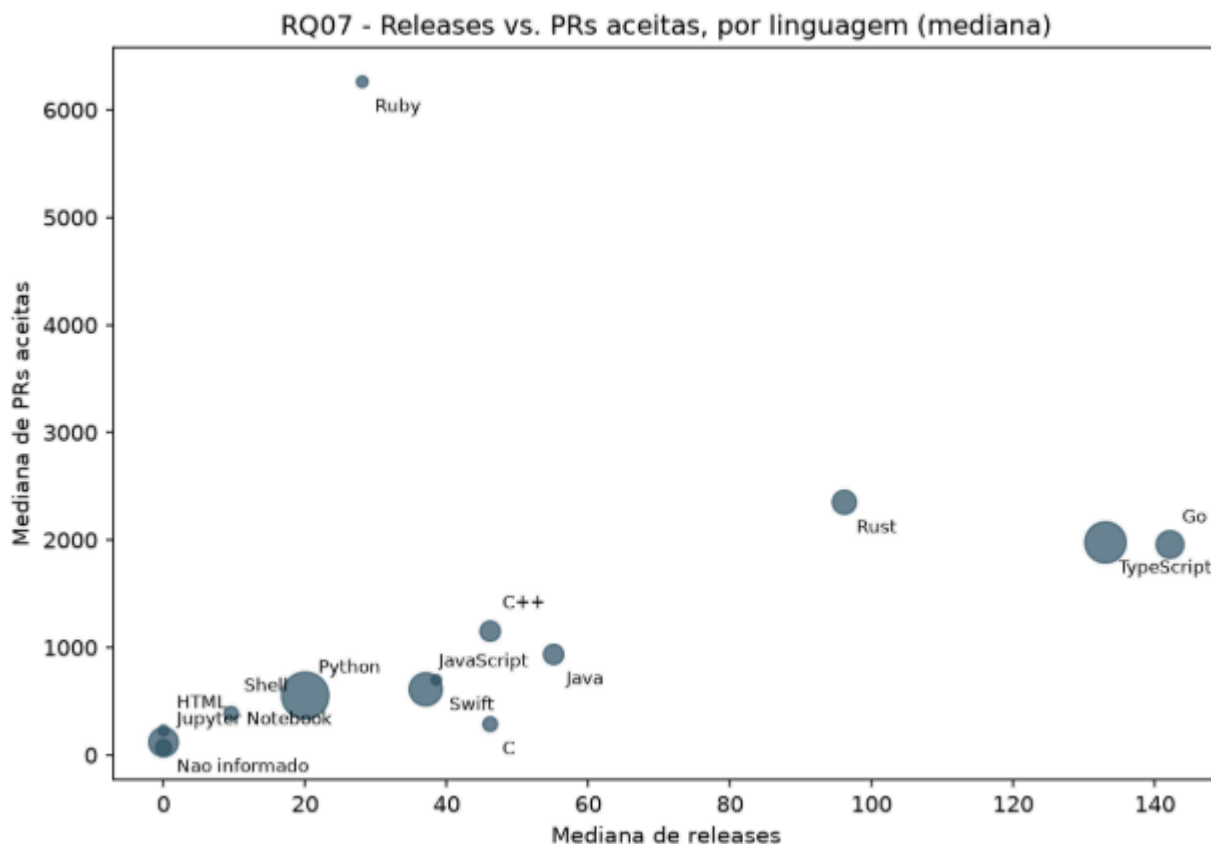


Amostra de 1.000 repositórios: idade mediana de 7,8 anos (média de 7,7 anos), variando de 11 dias a 18,4 anos. A distribuição não é concentrada num único período — há repositórios populares de praticamente todas as idades, com leve predominância entre 3 e 13 anos.



Amostra de 1.000: mediana de 768 pull requests aceitas, média de 4.243 (bem acima da mediana, puxada por poucos projetos com dezenas de milhares de PRs). 20 repositórios (2%) têm 0 PRs aceitas, apesar de populares — o outlier mais notável é o torvalds/linux, mirror read-only cujo desenvolvimento real acontece fora do GitHub.

RQ07 — Sistemas em linguagens populares recebem mais contribuição, lançam mais releases e são atualizados com mais frequência?



Considerando linguagens com pelo menos 10 repositórios na amostra: TypeScript e Go se destacam com medianas altas tanto em releases (133 e 142) quanto em PRs aceitas (1.979 e 1.958). Ruby é um caso à parte — mediana de 6.269 PRs aceitas com apenas 28 releases, mas é o grupo mais pequeno (13 repositórios), então esse valor deve ser lido com cautela. Python, apesar de ser a linguagem mais frequente na amostra (227 repositórios), tem medianas comparativamente baixas (20 releases, 560 PRs) — ou seja, ser a linguagem mais comum entre os populares não implica ter o maior volume de contribuição por repositório.

4.3 Discussão

RQ01 — hipótese informal (Issue #10): esperávamos que sistemas populares fossem majoritariamente maduros/antigos, por efeito de acumulação orgânica de estrelas ao longo do tempo. Resultado: parcialmente confirmada. A mediana de 7,8 anos indica maturidade predominante, mas a distribuição é ampla — repositórios com poucos meses de existência também aparecem entre os mais populares, sugerindo que maturidade ajuda mas não é pré-requisito para popularidade no GitHub.

RQ02 — hipótese informal (Issue #10): esperávamos que sistemas populares recebessem, em geral, muita contribuição externa. Resultado: confirmada, com uma exceção relevante. A mediana de 768 PRs aceitas sustenta a hipótese, mas 2% dos repositórios (incluindo o próprio kernel Linux) têm zero PRs aceitas no

GitHub — nesses casos, popularidade não se traduz em contribuição via Pull Request porque o desenvolvimento real acontece fora da plataforma.

RQ07 — hipótese informal: esperávamos que repositórios em linguagens mais populares (ex.: as do topo do GitHub Octoverse) também recebessem mais contribuição, lançassem mais releases e fossem atualizados com mais frequência que a média. Resultado: parcialmente confirmada. TypeScript e Go, linguagens bem posicionadas no Octoverse, de fato têm medianas altas nos três indicadores. Mas Python — a linguagem mais popular tanto no Octoverse quanto nesta amostra — tem medianas comparativamente modestas de releases e PRs aceitas, e Ruby aparece com mediana de PRs muito alta apesar de não estar entre as linguagens mais populares do mercado. Ou seja, popularidade da linguagem no ecossistema geral não é um preditor direto e linear do nível de contribuição externa recebida pelos repositórios individuais.

Ameaças à validade: (i) os dados refletem um instante único (snapshot) da coleta — número de estrelas, PRs e issues mudam continuamente, então os 1.000 repositórios "mais populares" de hoje não são exatamente os mesmos de amanhã; (ii) a API de busca do GitHub limita resultados a 1.000 itens, então repositórios entre a posição 1.001 e além ficam necessariamente fora da amostra, mesmo que estejam próximos em número de estrelas dos últimos incluídos; (iii) a fonte usada para "linguagem mais popular" (GitHub Octoverse) mede popularidade entre todos os repositórios públicos, não apenas os mais populares — a comparação com RQ05/RQ07 pressupõe que essa referência geral é aplicável ao recorte de repositórios de alta visibilidade, o que pode não se sustentar integralmente; (iv) `primaryLanguage` reflete a linguagem com mais bytes de código no repositório, não necessariamente a linguagem mais relevante para quem usa o projeto (ex.: repositórios de documentação classificados por uma linguagem de marcação).

5. Conclusão

O estudo confirma, de forma geral, que repositórios populares do GitHub compartilham características observáveis, mas com exceções relevantes que impedem generalizações simples. Quanto à maturidade (RQ01), há predominância de repositórios com vários anos de existência, mas uma minoria não desprezível alcança grande popularidade rapidamente — maturidade ajuda, mas não é condição necessária. Quanto à contribuição externa (RQ02) e à frequência de atualização (RQ04), a maioria dos repositórios populares de fato recebe contribuição ativa e é atualizada com frequência muito alta (mediana de poucos dias), confirmando a ideia de que visibilidade e atividade andam juntas — com a exceção notável de repositórios que são mirrors ou cujo desenvolvimento ocorre fora do GitHub. O uso de releases formais (RQ03) é mais heterogêneo: quase 30% dos repositórios populares não usam esse mecanismo, sugerindo que ele não é universal mesmo entre projetos de destaque. A composição por linguagem (RQ05) confirma a predominância das linguagens mais populares do mercado, com uma inversão específica (TypeScript à frente de JavaScript nesta amostra) que sugere que projetos de grande porte adotam tipagem estática mais que a média geral do GitHub. A saúde de manutenção, medida por issues fechadas (RQ06), é alta na maioria dos casos. Por fim, cruzando popularidade de linguagem com indicadores de contribuição (RQ07), a relação não é linear: linguagens populares como Python lideram em quantidade de repositórios, mas não necessariamente em intensidade de contribuição por repositório.

Como limitações principais, destacam-se: a natureza pontual da coleta (um único snapshot no tempo), o teto de 1.000 resultados imposto pela própria API do GitHub, e a dependência de uma única fonte externa (GitHub Octoverse) para avaliar popularidade de linguagem, que mede uma população mais ampla do que o recorte estudado aqui.

Com mais tempo, o grupo coletaria os dados em múltiplos pontos no tempo (ex.: mensalmente), permitindo observar tendências em vez de um retrato único, e armazenaria as métricas em nível de repositório (não só agregadas) desde a primeira coleta, facilitando análises adicionais sem precisar reprocessar os dados brutos.

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- GITHUB. Octoverse 2024: the state of open source. GitHub, 2024. Disponível em: <https://octoverse.github.com/>. Acesso em: 2026.